

CAGM

Compressed Arithmetic Gradient Machine

Technical Specification — Version 2.0

Classification: CONFIDENTIAL — Patent Pending

May 2026

This document contains proprietary and confidential information. Distribution, reproduction, or use outside of authorised parties is strictly prohibited.

1. Introduction

CAGM (Compressed Arithmetic Gradient Machine) is a fixed-point neural network classifier implemented entirely in integer arithmetic. It is designed for deterministic, reproducible training and inference across heterogeneous compute environments, eliminating floating-point non-determinism as a source of variance.

The system produces a cryptographic proof of its training trajectory alongside a trained classifier, enabling independent verification of a model's computational history without exposing model weights or raw training data.

All implementation-specific constants, initialisation seeds, and internal algorithm parameters are omitted from this specification and are protected under the pending patent filing.

1.1 Scope

This specification covers:

- System architecture and component responsibilities
- Data flow through training and inference pipelines
- Input/output interface and command-line contract
- Cryptographic proof and verification mechanism
- Concurrency model
- Numerical representation

It does not cover internal hyperparameter values, weight initialisation seeds, gradient clipping thresholds, or proof construction internals.

1.2 Intended Audience

This document is intended for patent agents, technical reviewers, and authorised engineering personnel evaluating the novelty and scope of the CAGM system.

2. System Overview

CAGM is a self-contained, single-binary classifier that accepts a CSV dataset and produces a trained model, an accuracy measurement, and a cryptographic proof of training integrity. The entire compute graph operates in fixed-point integer arithmetic with no floating-point operations in the training loop.

2.1 High-Level Architecture

The system is composed of five cooperating subsystems:

Subsystem	Responsibility
Data Loader	Parses CSV input; detects feature count and class labels automatically
Normalizer	Scales and standardises input features into a fixed-point representation; fit on training data, applied to all splits
CAGM Core (Neural Network)	Four-layer residual fixed-point neural network; performs forward pass, backpropagation, and Adam-variant weight updates
Integrity Engine (IE)	Canonicalises floating-point logits to a platform-independent bit pattern; provides pre/post-layer normalisation and logit hashing
Proof Builder	Accumulates cryptographic hashes of weights, gradients, and activations across training epochs; produces a final verifiable proof string

3. Data Pipeline

3.1 Input Format

Input must be a UTF-8 CSV file with a header row. All columns except the last are treated as numeric features. The last column is the integer class label (zero-based). At least two columns and four data rows are required.

3.2 Preprocessing

The Normalizer fits on the training split and applies a two-stage scaling procedure to each feature column:

- Stage 1 — Distribution-Aware Scaling: A per-feature scale factor is computed to bring the value range into the representable fixed-point space while respecting a target standard deviation. Scale bounds are enforced to prevent degenerate scaling.
- Stage 2 — Standardisation: Each feature is mean-centred and divided by its standard deviation after stage-1 scaling. The resulting value is quantised to a fixed-point integer.

The exact scale bounds, target standard deviation, and quantisation precision are internal parameters and are not disclosed in this document.

3.3 Train / Validation Split

The dataset is partitioned into training and validation subsets using stratified sampling to preserve class proportions. A deterministic pseudo-random shuffle is applied per class before splitting. The validation fraction is fixed internally.

For small datasets, effective epochs, patience, batch size, and learning rate are automatically adjusted by the system to compensate for limited sample size.

4. Neural Network Architecture

4.1 Layer Structure

The CAGM Core is a four-layer fully-connected neural network with residual skip connections:

Layer	Type	Input	Output	Post-processing
L1	Linear + Bias	n_features	hidden_dim	Layer Norm, Leaky ReLU
L2	Linear + Bias	hidden_dim	hidden_dim	Layer Norm, Leaky ReLU, Residual Add (L1 output)
L3	Linear + Bias	hidden_dim	hidden_dim	Layer Norm, Leaky ReLU, Residual Add (L2 output)
L4	Linear + Bias	hidden_dim	n_classes	Raw logits (no activation)

Residual connections are applied additively after activation of layers L2 and L3, carrying the output of the preceding hidden layer forward. This mitigates vanishing gradients without adding learnable parameters.

4.2 Fixed-Point Arithmetic

All weights, biases, activations, and gradient accumulators are stored as 32-bit signed integers representing values in a fixed-point format. Intermediate products use 64-bit accumulators to prevent overflow before reduction. Values are clamped to the representable range at each operation boundary.

The specific fractional bit-width (scaling factor) is an internal parameter and is not disclosed here.

4.3 Activation Function

Hidden layers use a Leaky ReLU activation. For non-positive inputs, the output is attenuated by a fixed-point alpha coefficient. The coefficient is an internal parameter.

4.4 Layer Normalisation

A batch-mode layer normalisation is applied after each linear transform of hidden layers L1–L3. For each sample independently, the mean is subtracted and the result is divided by the standard deviation plus a small epsilon, all computed in fixed-point. This stabilises training without requiring stored running statistics.

4.5 Weight Initialisation

Weights are initialised using a He-normal scheme: values are drawn from a normal distribution with mean 0 and standard deviation $\sqrt{2 / \text{fan_in}}$. The distribution is generated using a deterministic linear congruential generator (LCG) with a Box-Muller transform. Each weight matrix uses a distinct fixed seed. Biases are initialised to zero.

Initialisation seeds are internal constants and are not disclosed in this document.

4.6 Softmax

During the forward pass for training, class probabilities are derived from logits via a numerically stable fixed-point softmax. Logits are shifted by their per-sample maximum before exponentiation. Exponentiation uses a precomputed lookup table spanning a defined input range, evaluated at training initialisation.

5. Training Procedure

5.1 Optimiser

Weights are updated using an Adam-variant optimiser operating in fixed-point. The optimiser maintains first-moment (momentum) accumulators as fixed-point integers and second-moment (variance) accumulators as double-precision floats. Bias correction is applied using precomputed correction factors derived from the beta parameters. Weight decay (L2 regularisation) is folded into the weight gradient before the update step.

Beta coefficients, epsilon, and weight-decay values are internal parameters not disclosed here.

5.2 Learning Rate Schedule

Training uses a two-phase learning rate schedule:

- Warmup Phase: The learning rate increases linearly from zero to the configured start value over the warmup epoch count.
- Cosine Decay Phase: After warmup, the learning rate follows a cosine annealing curve, decaying smoothly to the configured end learning rate.

5.3 Mini-Batch Training

At each epoch, training samples are shuffled using a deterministic LCG-based Fisher-Yates shuffle seeded by the epoch index. Mini-batches are drawn sequentially from the shuffled order. The batch size is user-configurable and may be reduced automatically for small datasets.

5.4 Early Stopping

Validation accuracy is evaluated after each epoch. The best-performing weight snapshot is stored. Training terminates when validation accuracy fails to improve for a configurable patience count. At termination, the best-epoch weights are restored to the model before inference and proof generation.

5.5 Parallelism

Matrix multiplication is parallelised across multiple POSIX threads. The weight matrix rows are partitioned into equal chunks, each processed by one thread. Threads operate on disjoint output regions and require no synchronisation during the compute phase. The number of worker threads is user-configurable up to a compiled-in maximum.

6. Integrity Engine (IE)

The Integrity Engine provides platform-independent canonicalisation of floating-point logit values, ensuring that the cryptographic proof is identical across architectures that may represent special float values (NaN, negative zero) differently.

6.1 Floating-Point Canonicalisation

All logit float values are canonicalised before hashing. The canonicalisation procedure:

- Maps IEEE 754 NaN bit patterns to a canonical quiet-NaN representation.
- Maps negative zero to positive zero.

- Leaves all other values unchanged.

6.2 Correction Pass

A correction pass is applied to logit arrays at defined pipeline points. The pass maps each float to a canonical integer-aligned representation using a monotonicity-preserving bit reordering, snaps it to the nearest quantisation step, then inverts the mapping. This ensures that minor floating-point differences arising from different compiler optimisations or hardware FPU behaviour do not produce divergent hashes.

The step size used for quantisation snapping is an internal parameter.

6.3 Logit Hash

After canonicalisation and correction, the logit array is hashed using a non-cryptographic hash function (FNV-32a variant) applied to the corrected bit patterns. This produces a compact 32-bit fingerprint for cross-platform verification.

6.4 IE Lifecycle

The IE is applied at three defined points in the inference pipeline:

- Entry: Canonicalise raw floats on input.
- Pre-layer: Apply correction pass before each layer.
- Post-layer: Apply correction pass after each layer; final correction and re-canonicalisation applied at the last layer boundary.

7. Cryptographic Proof System

CAGM generates a 64-character hexadecimal proof string that cryptographically binds the trained model to its training trajectory. The proof enables an independent party to verify that a given model is the authentic result of a specific training run on specific data, without access to the raw source data or full weight history.

7.1 Hash Primitive

All cryptographic hashing uses SHA-256, implemented as a self-contained, portable C function with no external library dependency. Combining two hash values is performed by concatenating the two 32-byte digests and re-hashing, producing a single 32-byte output.

7.2 Training Trajectory Components

The proof incorporates hashes from four independent trace streams accumulated during training:

Trace	What is Hashed	When
Weight Trace	SHA-256 of all weight and bias arrays concatenated	After each epoch
Gradient Trace	Combined SHA-256 of each weight-gradient array (hashed individually, then combined)	After each mini-batch
Activation Trace	Rolling combination of per-batch activation hashes	After each mini-batch; accumulated per epoch
Gradient Chain	Blockchain-style chain: each epoch commits a hash of {epoch index, activation hash, combined gradient hash, previous chain hash}	After each epoch

7.3 Merkle Aggregation

At the end of training, each trace stream is reduced to a single root hash using a Merkle tree construction. Leaf nodes are the per-epoch or per-batch hashes. The tree pads to an even count by duplicating the last node when necessary. The root is computed by iteratively hashing adjacent pairs until one root hash remains.

7.4 Proof Assembly

The final proof string is a SHA-256 hash derived from a structured combination of:

- A versioned prefix string identifying the proof scheme.
- A SHA-256 hash of the raw (pre-normalisation) training data.
- A SHA-256 hash of the normalised (post-preprocessing) training data.
- A SHA-256 hash of the final trained weight snapshot.
- The Merkle root of the weight trace.
- The Merkle root of the gradient trace.
- The Merkle root of the activation trace.
- The final gradient chain hash.
- SHA-256 hashes of the final weight and bias arrays.

The combination procedure uses the same pairwise SHA-256 combine primitive throughout, producing a single 32-byte digest that is hex-encoded as the output proof string.

The exact ordering and concatenation scheme for proof assembly is an internal detail not disclosed in this specification.

7.5 Verification

A third party in possession of the same dataset and the binary can reproduce the proof by rerunning training with identical hyperparameters. The proof is fully deterministic: the same data, configuration, and binary will always produce the same 64-character proof string.

8. Command-Line Interface

8.1 Invocation

```
cagm <data.csv> [options]
```

8.2 Arguments

Argument / Option	Type	Default	Description
<data.csv>	positional	—	Path to input CSV file. Last column = class label (integer, 0-based). All other columns = numeric features.
--hidden <int>	optional	64	Hidden layer width (number of neurons per hidden layer).
--epochs <int>	optional	100	Maximum training epochs.
--batch <int>	optional	64	Mini-batch size. Auto-reduced for small datasets.
--lr <float>	optional	0.005	Initial (peak) learning rate.
--lr-end <float>	optional	0.0003	Final learning rate at end of cosine decay.
--warmup <int>	optional	8	Number of warmup epochs for linear LR ramp.
--patience <int>	optional	20	Early-stopping patience (epochs without val improvement).
--threads <int>	optional	4	Number of worker threads (1–16).
--verbose	flag	off	Print per-epoch training progress to stderr.
--ie-test	flag	off	Run IE canonicalisation smoke-test and exit.
--help / -h	flag	—	Print usage message and exit.

8.3 Standard Output

On successful completion, the program writes three lines to stdout:

Key	Format	Description
accuracy	accuracy <float>	Training-set accuracy of the restored best-epoch model (0.0–1.0, 6 decimal places).
proof	proof <hex64>	64-character lowercase hex SHA-256 proof of training trajectory.
ie_verify	ie_verify <PASS FAIL> (hash=0x<hex8>)	Result of Integrity Engine round-trip verification on a logit sample from the first training row.

Diagnostic and progress messages are written to stderr only and are not part of the machine-readable output contract.

8.4 IE Smoke-Test Mode

When invoked with `--ie-test`, the binary exercises the Integrity Engine on a synthetic float array containing representative edge cases (positive values, negative values, NaN, negative zero) and reports the computed hash and verification result. This mode exits without reading any CSV file.

9. Numerical Stability and Determinism

9.1 Determinism Guarantees

Given identical inputs and configuration, CAGM produces bit-identical outputs across runs and across platforms. This guarantee is achieved by:

- Using integer arithmetic exclusively in the training loop, eliminating floating-point reordering effects.
- Using a deterministic LCG for all pseudo-random operations (shuffles, initialisations), with no dependence on OS entropy.
- Applying the Integrity Engine to canonicalise any floating-point values that cross an interface boundary.
- Accumulating intermediate products in 64-bit integers before reduction, preventing platform-dependent rounding.

9.2 Overflow Handling

All fixed-point values are clamped to a defined representable range at each operation boundary. Gradient values are additionally clipped to a separate, narrower bound before accumulation. This prevents integer overflow without floating-point fallback.

9.3 Normaliser Stability

Features with near-zero standard deviation are handled without division by zero: a minimum standard deviation floor is enforced during fit. Scale factors are bounded above and below to prevent degenerate transformations on outlier-heavy features.

10. Security and Confidentiality Properties

The proof system provides the following security properties:

10.1 Training Integrity

The proof binds the output model to a specific sequence of weight updates. Any alteration of the training data, hyperparameters, or binary (absent re-computation) will produce a different proof. The proof cannot be forged without access to the full training trajectory.

10.2 Data Privacy

The proof does not expose raw training samples or individual feature values. The raw-data hash component commits to the dataset without revealing it; verification requires possession of the original data.

10.3 Collision Resistance

Proof security inherits from SHA-256 collision resistance. The probability of two distinct training trajectories producing the same proof is negligible under standard cryptographic assumptions.

10.4 Chain Linkage

The gradient chain incorporates the hash of the preceding epoch as an input to each epoch commit. This blockchain-style linkage ensures that the proof reflects the full sequential order of training updates and cannot be reordered or truncated without detection.

11. Build Dependencies

Dependency	Purpose	Notes
C Standard Library (C99)	Memory allocation, string handling, stdio	No external C++ or C11+ features required
POSIX Threads (pthreads)	Parallel matrix multiplication	Available on Linux, macOS, and POSIX-compliant platforms
libm	sqrt, cos, exp, log, round (init only)	Only used outside the deterministic training loop

Dependency	Purpose	Notes
SHA-256	Cryptographic hashing for proof	Fully self-contained implementation; no OpenSSL or similar dependency

No third-party machine learning frameworks, BLAS libraries, or GPU runtimes are required. The binary compiles with standard C99 toolchains.

12. Glossary

Term	Definition
CAGM	Compressed Arithmetic Gradient Machine — the neural network classifier system described in this document.
Fixed-point	A numeric representation where a 32-bit integer encodes a real number at a fixed fractional precision, avoiding floating-point hardware.
LCG	Linear Congruential Generator — a deterministic pseudo-random number generator used for shuffles and weight initialisation.
Merkle tree	A binary hash tree where each internal node is the hash of its two children, used to produce a single digest representing a sequence of values.
IE	Integrity Engine — the subsystem that canonicalises floating-point logits to platform-independent bit patterns.
Proof	The 64-character hexadecimal SHA-256 string that cryptographically summarises the complete training trajectory.
Residual connection	A skip connection that adds the output of a preceding layer directly to the output of a later layer, bypassing intermediate transforms.
Stratified split	A dataset partition method that preserves the proportion of each class label in both the training and validation subsets.
Adam	An adaptive gradient optimiser that maintains per-parameter first and second moment estimates to adjust effective learning rates.
Gradient Chain	A blockchain-style linked sequence of per-epoch hashes that records the order and content of all gradient updates.

END OF SPECIFICATION

CAGM v2 — Proprietary & Patent Pending — All rights reserved